

PCAD™ – Certified Associate Data Analyst with Python EXAM SYLLABUS



(Exam PCAD-31-02)

Last revised: July 14, 2025

1. Data Acquisition and Pre-Processing (14)

1.1 Data Collection, Integration, and Storage (3)

1.1.1 Explain and compare data collection methods and their use in research, business, and analytics.

- A. Explore different techniques: Surveys, interviews, web scraping.
- B. Discuss representative sampling, challenges in data collection, and differences between qualitative and quantitative research.
- C. Examine legal and ethical considerations in data collection.
- D. Explain the importance of data anonymization in maintaining privacy and confidentiality, particularly with personally identifiable information (PII).
- E. Investigate the impact of data collection on business strategy formation, market research accuracy, risk assessment, policy-making, and business decisions.
- F. Explain the process and methodologies of data collection, including survey design, audience selection, and structured interviews.

1.1.2 Aggregate data from multiple sources and integrate them into datasets.

- A. Explain techniques for combining data from various sources, such as databases, APIs, and file-based storage.
- B. Address challenges in data aggregation, including data format disparities and alignment issues.
- C. Understand the importance of data consistency and accuracy in aggregated datasets.

1.1.3 Explain various data storage solutions.

- A. Understand various data storage methods and their appropriate applications.
- B. Distinguish between the concepts of data warehouses, data lakes, and file-based storage options like CSV and Excel.
- C. Explain the concepts of cloud storage solutions and their growing role in data management.

1.2 Data Cleaning and Standardization (4)

1.2.1 Understand structured and unstructured data and their implications in data analysis.

- A. Recognize the characteristics of structured data, such as databases and spreadsheets, and their straightforward use in analysis.
- B. Understand unstructured data, including text, images, and videos, and the additional processing required for analysis.
- C. Explore how the data structure impacts data storage, retrieval, and analytical methods.

1.2.2 Identify, rectify, or remove erroneous data.

- A. Identify data errors and inconsistencies through various diagnostic methods.
- B. Address missing, inaccurate, or misleading information.
- C. Tackle specific data quality issues: numerical data problems, duplicate records, invalid data entries, and missing values.
- D. Explain different types of missingness (MCAR, MAR, MNAR), and their implications for data analysis.
- E. Explore various techniques for dealing with missing data, including data imputation methods.
- F. Understand the implications of data correction or removal on overall data integrity and analysis outcomes.

- G. Explain the importance of data collection in the context of outlier detection.
- H. Explain why high-quality data is crucial for accurate outlier detection.
- I. Explain how different data types (numerical, categorical) may influence outlier detection strategies.

1.2.3 Understand data normalization and scaling.

- A. Understand the necessity of data normalization to bring different variables onto a similar scale for comparative analysis.
- B. Understand various scaling methods like Min-Max scaling and Z-score normalization.
- C. Explain encoding categorical variables for quantitative analysis, including one-hot encoding and label encoding methods.
- D. Explain the pros and cons of data reduction (reduce the number of variables under consideration or simplify the models vs loss of data explainability).
- E. Explain methods for handling outliers, including detection and treatment techniques to ensure data quality.
- F. Understand the importance of data format standardization across different datasets for consistency, especially when dealing with date-time formats and numerical values.

1.2.4 Apply data cleaning and standardization techniques.

- A. Perform data imputation techniques, string manipulation, data format standardization, boolean normalization, string case normalization, and string-to-number conversions.
- B. Discuss the pros and cons of imputation vs. exclusion and their impact on the reliability and validity of the analysis.
- C. Explain the concept of One-Hot Encoding and its application in transforming categorical variables into a binary format, and preparing data for machine learning algorithms.
- D. Explain the concept of bucketization and its application in transforming continuous variables into categorical variables.

1.3 Data Validation and Integrity (2)

1.3.1 Execute and understand basic data validation methods.

- A. Define “validation” (type, range, cross-field) and match them to tools (Python logic, schema checks).
- B. Perform type, range, and cross-reference checks.
- C. Explain the benefit of early type checks in ingestion scripts.

1.3.2 Establish and maintain data integrity through clear validation rules.

- A. Understand the concept of data integrity and its importance in maintaining reliable and accurate databases.
- B. Apply clear validation rules that enforce the correctness and consistency of data.

1.4 Data Preparation Techniques (5)

1.4.1 Understand File Formats in Data Acquisition.

- A. Explain the roles and characteristics of common data file formats: CSV for tabular data, JSON for structured data, XML for hierarchically organized data, and TXT for unstructured text.
- B. Understand basic methods for importing and exporting these file types in data analysis tools, focusing on practical applications.

1.4.2 Access, manage, and effectively utilize datasets.

- A. Understand the basics of accessing datasets from various sources like local files, databases, and online repositories.
- B. Understand the principles of data management, including organizing, sorting, and filtering data in preparation for analysis.

1.4.3 Extract data from various sources.

- A. Explain fundamental techniques for extracting data from various sources, emphasizing methods to retrieve and collate data from databases, APIs, and online services.
- B. Extract data from HTML using Python tools and libraries (BeautifulSoup, requests)
- C. Understand basic challenges and considerations in data extraction, such as data compatibility and integrity.
- D. Discuss ethical web scraping practices, including respect for robots.txt and rate-limiting.

1.4.4 Apply spreadsheet best practices for readability and formatting.

- A. Improve the readability and usability of data in spreadsheets, focusing on layout adjustments, formatting best practices, and basic formula applications.

1.4.5 Prepare, adapt, and pre-process data for analysis.

- A. Understand the importance of the surrounding context, objectives and stakeholder expectations to guide the preparation steps.
- B. Understand basic concepts of data pre-processing, including sorting, filtering, and preparing data sets for analytical work.
- C. Discuss the importance of proper data formatting for analysis, such as ensuring consistency in date-time formats and aligning data structures.
- D. Introduce concepts of dataset structuring, including the basics of transforming data into a format suitable for analysis (e.g., wide vs. long formats).
- E. Explain the concept of splitting data into training and testing sets, particularly for machine learning projects, emphasizing the importance of this step for model validation.
- F. Understand the impact of outlier management on data quality in preprocessing.

2. Programming and Database Skills (16)

2.1 Core Python Proficiency (5)

2.1.1 Apply Python syntax and control structures to solve data-related problems.

- A. Accurately use basic Python syntax for variables, scopes, and data types.
- B. Implement control structures like loops and conditionals to manage data flow.

2.1.2 Analyze and create Python functions.

- A. Design functions with clear purpose, using both indexed and keyword arguments.
- B. Differentiate between optional and required arguments and apply them effectively.

2.1.3 Evaluate and navigate the Python Data Science ecosystem.

- A. Identify key Python libraries and tools essential for data science tasks.
- B. Critically assess the suitability of various Python resources for different data analysis scenarios.

2.1.4 Organize and manipulate data using Python's core data structures.

- A. Effectively use tuples, sets, lists, dictionaries, and strings for data organization and manipulation.

- B. Solve complex data handling tasks by choosing appropriate data structures.

2.1.5 Explain and implement Python scripting best practices.

- A. Understand and apply PEP 8 guidelines for Python coding style.
- B. Comprehend and utilize PEP 257 for effective docstring conventions to enhance code documentation.

2.2 Module Management and Exception Handling (2)

2.2.1 Import modules and manage Python packages using PIP.

- A. Apply different types of module imports (standard imports, selective imports, aliasing)
- B. Understand importing modules from different sources (Python Standard Library, via package managers like PIP, and from locally developed modules/packages).
- C. Identify and import necessary Python modules for specific tasks, understanding the functionality and purpose of each.
- D. Demonstrate proficiency in managing Python packages using PIP, including installing, updating, and removing packages.

2.2.2 Apply basic exception handling and maintain script robustness.

- A. Implement basic exception handling techniques to manage and respond to errors in Python scripts.
- B. Predict common errors in Python code and develop strategies to handle them effectively.
- C. Interpret error messages to diagnose and resolve issues, enhancing the robustness and reliability of Python scripts.

2.3 Object-Oriented Programming for Data Modeling (3)

2.3.1 Apply basic object-oriented programming to structure and model data.

- A. Define and instantiate classes that represent structured data records, including constructors and instance variables.
- B. Organize attributes and behaviors within objects using constructors and instance methods.
- C. Apply encapsulation principles by using naming conventions (e.g., `__protected`, `__private`) and method-based access (getters and setters) to manage internal object state and support clean design.

2.3.2 Apply object-oriented patterns to enhance code reuse and clarity in analysis workflows.

- A. Use composition to group related data models (e.g., nesting a *User* object inside a *Response* object).
- B. Extend base classes using inheritance and override methods for specialized behavior (e.g., multiple exporter classes).
- C. Demonstrate polymorphism by calling the same method (e.g., *.process()*, *.export()*) on different subclasses within a data workflow.

2.3.3 Manage object identity and comparisons in data pipelines.

- A. Use reference variables and understand shared vs independent object behavior (e.g., mutation of lists inside objects).
- B. Compare object content using `==`, `is`, and implement custom equality with `__eq__()`.

2.4 SQL for Data Analysts (6)

2.4.1 Perform SQL queries to retrieve and manipulate data.

- A. Compose and execute SQL queries to extract data from database tables.
- B. Apply SQL functions and clauses to manipulate and filter data effectively.
- C. Construct and execute SQL queries using SELECT, FROM, JOINS (INNER, LEFT, RIGHT, FULL), WHERE, GROUP BY, HAVING, ORDER BY, and LIMIT.
- D. Analyze data retrieval needs and apply appropriate clauses from the SFJWGHOL set to meet those requirements effectively.

2.4.2 Execute fundamental SQL commands to create, read, update, and delete data in database tables.

- A. Demonstrate the ability to use CRUD operations (Create, Read, Update, Delete) in SQL.
- B. Construct SQL statements for data insertion, retrieval, updating, and deletion.

2.4.3 Establish connections to databases using Python.

- A. Understand and implement methods to establish database connections using Python libraries (e.g., `sqlite3`, `pymysql`).
- B. Analyze and resolve common issues encountered while connecting Python scripts to databases.

2.4.4 Execute parameterized SQL queries through Python to safely interact with databases.

- A. Develop and execute parameterized SQL queries in Python to interact with databases securely.
- B. Evaluate the advantages of parameterized queries in preventing SQL injection and maintaining data integrity.

2.4.5 Understand, manage and convert SQL data types appropriately within Python scripts.

- A. Identify and understand various SQL data types and their counterparts in Python.
- B. Practice converting data types appropriately when transferring data between SQL databases and Python scripts.

2.4.6 Understand essential database security concepts, including strategies to prevent SQL query injection.

- A. Comprehend fundamental database security principles, including measures to prevent SQL injection attacks.
- B. Assess and apply strategies for writing secure SQL queries within Python environments.

3. Statistical Analysis (4)

3.1 Descriptive Statistics (2)

3.1.1 Understand and apply statistical measures in data analysis.

- A. Understand and describe measures of central tendency and spread.
- B. Identify fundamental statistical distributions (Gaussian, Uniform) and interpret their trends in various contexts (over time, univariate, bivariate, multivariate).
- C. Apply confidence measures in statistical calculations to assess data reliability.

3.1.2 Analyze and evaluate data relationships.

- A. Analyze datasets to identify outliers and evaluate negative and positive correlations using Pearson's R coefficient.

- B. Interpret and critically assess information presented in various types of plots and graphs, including Boxplots, Histograms, Scatterplots, Lineplots, and Correlation heatmaps.

3.2 Inferential Statistics (2)

3.2.1 Understand and apply bootstrapping for sampling distributions.

- A. Understand the theoretical basis and statistical principles underlying bootstrapping.
- B. Differentiate between discrete and continuous data types in the context of bootstrapping.
- C. Recognize situations and data types where bootstrapping is an effective method for estimating sampling distributions.
- D. Demonstrate proficiency in applying bootstrapping methods using Python to generate and analyze sampling distributions.
- E. Analyze the reliability and validity of results obtained from bootstrapping in various statistical scenarios.

3.2.2 Explain when and how to use linear and logistic regression, including appropriateness and limitations.

- A. Comprehend the theory, assumptions, and mathematical foundation of linear regression.
- B. Explain the concepts, use cases, and statistical underpinnings of logistic regression.
- C. Develop the ability to choose between linear and logistic regression based on the nature of the data and the research question.
- D. Apply the concepts of discrete and continuous data in choosing and implementing linear and logistic regression models.
- E. Demonstrate the application of linear and logistic regression models on datasets using Python, including parameter estimation and model fitting.
- F. Accurately interpret the outcomes of regression analyses, including coefficients and model fit statistics.
- G. Identify limitations, assumptions, and potential biases in linear and logistic regression models and their impact on results.

4. Data Analysis and Modeling (9)

4.1 Data Analysis with Pandas and NumPy (6)

4.1.1 Organize and clean data using Pandas

- A. Use Pandas to filter, sort, and manage missing or inconsistent values in tabular datasets.
- B. Prepare raw data for analysis by applying foundational data cleaning techniques.

4.1.2 Merge and reshape datasets using Pandas

- A. Apply advanced data manipulation techniques such as merging, joining, pivoting, and reshaping data frames.
- B. Structure datasets appropriately to support specific analysis workflows.

4.1.3 Understand the relationship between Series and DataFrames

- A. Explain the conceptual differences and connections between Pandas Series and DataFrames.
- B. Use indexing techniques and vectorized functions to navigate and transform data.

4.1.4 Access and manipulate data using locators and slicing

- A. Retrieve and modify data accurately using `.loc`, `.iloc`, slicing, and conditional selection.
- B. Apply indexing strategies to ensure efficient and accurate data access.

4.1.5 Perform array operations and distinguish between core data structures

- A. Use NumPy to execute array-based operations including arithmetic, broadcasting, and aggregations.
- B. Differentiate between arrays, lists, Series, DataFrames, and NDArrays, and evaluate their use cases and performance.

4.1.6 Group, summarize, and extract insights from data

- A. Group data using `groupby()` and create summary tables using pivot and cross-tabulation techniques.

- B. Calculate descriptive statistics using Pandas and NumPy to identify trends, detect anomalies, and support decision-making.

4.2 Statistical Methods and Machine Learning (3)

4.2.1 Apply Python's descriptive statistics for dataset analysis.

- A. Calculate and interpret key statistical measures such as mean, median, mode, variance, and standard deviation using Python.
- B. Utilize Python libraries (like Pandas and NumPy) to generate and analyze descriptive statistics for real-world datasets.

4.2.2 Recognize the importance of test datasets in model evaluation.

- A. Understand the role of test datasets in validating the performance of machine learning models.
- B. Demonstrate knowledge of proper test dataset selection and usage to ensure unbiased and accurate model evaluation.

4.2.3 Analyze and Evaluate Supervised Learning Algorithms and Model Accuracy.

- A. Analyze various supervised learning algorithms to understand their specific characteristics and applications.
- B. Evaluate the concepts of overfitting and underfitting within these models, including a detailed explanation of the bias-variance tradeoff.
- C. Assess the intrinsic tendencies of linear and logistic regression in relation to this tradeoff, and apply this understanding to prevent model accuracy issues.

5. Data Communication and Visualization (5)

5.1 Data Visualization Techniques (3)

5.1.1 Demonstrate essential proficiency in data visualization with Matplotlib and Seaborn.

- A. Utilize Matplotlib and Seaborn to create various types of plots, including Boxplots, Histograms, Scatterplots, Lineplots, and Correlation heatmaps.

- B. Interpret the data and findings represented in these visualizations to gain deeper insights and communicate results effectively.

5.1.2 Assess the pros and cons of different data representations.

- A. Evaluate the suitability of various chart types for different types of data and analysis objectives.
- B. Critically analyze the effectiveness of chosen visualizations in conveying the intended message or insight.

5.1.3 Label, annotate, and refine data visualizations for clarity and insight

- A. Incorporate labels, titles, and annotations in visualizations to clarify and emphasize key insights.
- B. Utilize visual exploration to generate hypotheses and test insights from datasets.
- C. Practice making data-driven decisions based on the interpretation of visualized data.
- D. Customize colors in plots to improve readability of a scatterplot.
- E. Label axes and add titles to improve data readability.
- F. Manipulate legend properties such as position, font size, and background color, to improve the esthetics and readability of data.

5.2 Effective Communication of Data Insights (2)

5.2.1 Tailor communication to different audience needs, and combine visualizations and text for clear data presentation.

- A. Analyze the audience to understand their background, interests, and knowledge level.
- B. Adapt communication style and content to meet the specific needs and expectations of diverse audiences.
- C. Create presentations and reports that effectively convey data insights to both technical and non-technical stakeholders.
- D. Integrate visualizations seamlessly into presentations and reports, aligning them with the narrative.
- E. Use concise and informative text to complement visualizations, providing context and key takeaways.
- F. Ensure visual and textual elements work harmoniously to enhance data clarity and understanding.
- G. Avoid slide clutter and optimize slide content to maintain focus on key messages.

- H. Craft a compelling data narrative that tells a story with data, highlighting insights and actionable takeaways.
- I. Select an appropriate and consistent color palette for visualizations, ensuring clarity and accessibility.

5.2.2 Summarize key findings and support claims with evidence and reasoning.

- A. Understand the process of identifying and extracting key findings from data analysis.
- B. Apply techniques to condense complex information into concise and meaningful summaries.
- C. Prioritize and emphasize the most relevant insights based on context.
- D. Explain the importance of backing assertions and conclusions with data-driven evidence and reasoning.
- E. Articulate the basis for claims and recommendations, demonstrating transparency in decision-making.
- F. Demonstrate proficiency in clearly presenting evidence to support claims and recommendations.

MQC Profile

A Minimally Qualified Candidate (MQC) for the *PCAD™ – Certified Associate Data Analyst with Python* certification is expected to demonstrate the essential skills and knowledge required to support entry-level data analysis tasks in professional settings.

The candidate should understand how to acquire, clean, prepare, analyze, model, and communicate data using Python, SQL, and widely used data tools and libraries. They must be able to connect to data sources, including databases, spreadsheets, APIs, and HTML web pages, and collect relevant data using tools and libraries such as `requests` and `BeautifulSoup`.

The MQC is proficient in writing and organizing Python scripts that utilize variables, functions, control flow, and data structures like lists, dictionaries, and sets. They apply best practices in Python programming, including documentation, error handling, and modular design, and are able to manage packages using `pip`.

They can use libraries such as Pandas, NumPy, and statistics to clean, reshape, and analyze structured datasets, as well as calculate descriptive statistics, correlations, and basic aggregations. They should also be able to execute SQL queries to retrieve and manipulate data and use sqlite3 to connect Python scripts to databases. They understand how to use parameterized queries to ensure data integrity and avoid SQL injection.

The candidate can perform basic statistical modeling, including linear and logistic regression, and apply inferential techniques such as bootstrapping. They should recognize the importance of model validation, test data splitting, and the risks of overfitting.

Finally, the MQC is able to create clear, insightful, and audience-appropriate visualizations using Matplotlib and Seaborn, structure effective data stories, and present findings in both written and verbal formats, using design and communication best practices.

Block 1: Data Acquisition and Pre-Processing

Weight: 29.2% of total exam (14 items)

Minimum Coverage:

The MQC understands what data is, how it is structured, and how it is transformed into usable information for analysis. They can describe different data types – including structured, semi-structured, and unstructured – and explain how these formats influence storage, processing, and analytical techniques. The MQC is able to explain and compare various data collection methods such as surveys, interviews, APIs, and web scraping using tools like BeautifulSoup, and understands how these methods are used in research, business, and analytics contexts.

They can identify appropriate storage options for different data types, including CSV, JSON, Excel, databases, data lakes, and warehouses, and explain the role of cloud-based storage in modern data ecosystems. They recognize how poor data collection or storage practices can lead to quality issues and errors later in the analytics process.

The MQC is capable of integrating data from multiple sources and resolving inconsistencies during aggregation. They understand the implications of format alignment, type mismatches, and schema discrepancies. They can apply basic data cleaning techniques, including identifying and correcting missing, duplicate, or invalid values. They understand the importance of encoding categorical data, scaling numerical values, and formatting date-time values for consistency.

They can apply basic validation methods (such as type checks, range checks, and cross-reference logic) to ensure data quality and integrity. They are also able to prepare data for analysis by sorting, filtering, reshaping (wide vs. long), and splitting data into training and testing sets, particularly in preparation for modeling tasks. The MQC also understands ethical and legal responsibilities when working with personal data, including anonymization, consent, and compliance with frameworks such as GDPR and HIPAA.

Block 2: Programming and Database Skills

Weight: 33.3% of total exam (16 items)

Minimum Coverage:

The MQC is proficient in Python and uses the language to support data processing tasks. They can define and manipulate variables, use core data types (integers, floats, strings, booleans), and work with fundamental data structures such as lists, dictionaries, tuples, and sets. They write clear and reusable code using functions with parameters and return values, and they apply control flow constructs such as conditional statements and loops to process and analyze data efficiently.

They demonstrate awareness of clean coding practices, including proper indentation, naming conventions, and documentation using PEP 8 and PEP 257 standards. They are familiar with Python's standard library modules – such as `csv`, `os`, `math`, `statistics`, `datetime`, and `collections` – and know how to install and manage third-party packages using `pip`.

The MQC is comfortable using object-oriented programming (OOP) concepts to structure and encapsulate data. They can define classes, create objects, and organize attributes and methods in a way that supports reusability and clarity in data workflows.

In addition to Python, the MQC is expected to use SQL to query and manipulate structured data. They can retrieve and filter data using `SELECT`, `WHERE`, and various types of `JOIN` clauses, and they are capable of aggregating and grouping data using `GROUP BY`, `HAVING`, and `ORDER BY`. They understand how to perform CRUD operations and can write SQL statements to insert, update, and delete data.

They can connect Python scripts to relational databases using libraries such as `sqlite3`, and they understand how to execute parameterized queries to protect against SQL injection and ensure data integrity. The MQC also understands how to convert data types appropriately between SQL and Python when extracting or inserting data.

Block 3: Statistical Analysis

Weight: 8.3% of total exam (4 items)

Minimum Coverage:

The MQC has a solid grasp of foundational statistical concepts and can apply descriptive statistics to summarize datasets. They understand and can calculate measures of central tendency (mean, median, mode) and dispersion (standard deviation, variance), and they can describe basic distribution types, such as normal and uniform distributions.

They can assess relationships between variables using correlation analysis, particularly Pearson's R, and can identify outliers both visually and statistically. The MQC understands that visualizations like histograms, boxplots, and scatterplots support exploratory understanding of data distributions and trends.

In terms of inferential statistics, the MQC is familiar with bootstrapping as a method for estimating sampling distributions, especially when theoretical distributions are unknown. They understand the difference between discrete and continuous data and recognize when bootstrapping is appropriate for evaluating reliability.

They can explain and apply linear and logistic regression, understand the assumptions behind these models, and interpret their outputs, including coefficients and model fit statistics. The MQC is also aware of common limitations, such as overfitting, and can discuss the importance of model validation.

Block 4: Data Analysis and Modeling

Weight: 18.8% of total exam (9 items)

Minimum Coverage:

The MQC is proficient in using Pandas and NumPy for data analysis. They can clean and organize data using functions like `dropna()`, `fillna()`, `sort_values()`, and `replace()`, and they can reshape and restructure data using methods such as `pivot()`, `melt()`, `groupby()`, and `merge()`. They understand the difference between `DataFrame` and `Series` objects and can use `.loc` and `.iloc` to access and modify data precisely.

Using NumPy, the MQC performs numerical operations, including element-wise calculations, aggregations, and array broadcasting. They understand the performance advantages of NumPy arrays over native Python lists for large datasets.

They can compute descriptive summaries and perform basic feature engineering tasks such as bucketing, scaling, and encoding in preparation for modeling. The MQC is capable of analyzing datasets with grouped summaries and conditional filters, using combinations of Pandas and NumPy functionality to extract insights.

They understand the structure and purpose of supervised learning workflows and can apply train/test splits to evaluate models. They are able to fit and interpret basic linear and logistic regression models and understand key modeling concepts such as accuracy, overfitting, underfitting, and the bias-variance tradeoff.

Block 5: Data Communication and Visualization

Weight: 10.4% of total exam (5 items)

Minimum Coverage:

The MQC is able to interpret and create effective visual representations of data using Matplotlib and Seaborn. They understand how to choose appropriate chart types based on data type and purpose, and can generate bar charts, histograms, scatterplots, boxplots, line charts, and correlation heatmaps to highlight trends and patterns.

They know how to enhance chart readability by adding labels, titles, legends, gridlines, and appropriate color schemes. They can annotate graphs to draw attention to key insights and customize chart aesthetics to improve clarity.

The MQC understands how to communicate findings to different audiences by combining visuals with concise and informative summaries. They can adapt their messaging to suit both technical stakeholders (e.g., data teams) and non-technical audiences (e.g., managers, clients).

They are able to structure a data narrative that connects analysis to business or research questions, ensuring that conclusions are supported by data and that recommendations are evidence-based. They demonstrate awareness of design principles for slide decks and written reports, and avoid clutter, ambiguity, and misleading visuals.

Passing Requirement

To pass the PCAD exam, a candidate must achieve a **cumulative average score of at least 75%** across all exam blocks.

PCAD-31-02 Exam Structure Summary

The PCAD™ exam consists of single-select and multiple-select items designed to evaluate a candidate's ability to acquire, process, analyze, model, and communicate data using Python and SQL. Each item is worth a maximum of 1 point. After the exam is completed, the candidate's raw score is normalized, and the final result is expressed as a percentage.

The exam is divided into five blocks, each covering a specific area of data analytics and core technical skills in programming and SQL querying. The distribution of items and their corresponding weights reflects the relative emphasis placed on each area in professional practice.

Block Number	Block Name	Number of Items	Weight
1	Data Acquisition and Pre-Processing	14	29.2%
2	Programming and Database Skills	16	33.3%
3	Statistical Analysis	4	8.3%
4	Data Analysis and Modeling	9	18.8%
5	Data Communication and Visualization	5	10.4%
	Total	48	100%